

Python para Análise e Automação de Dados

Material-base para estudo e uso como fonte no Gemini Notebook

1. Objetivo

Este material reúne conceitos introdutórios sobre Python, análise de dados e automação de tarefas com arquivos e planilhas.

O objetivo é servir como uma fonte estruturada para um segundo cérebro sobre Python para Análise e Automação de Dados.

2. Python

Python é uma linguagem de programação de propósito geral, com sintaxe relativamente simples e uma biblioteca padrão extensa.

A documentação oficial apresenta recursos da linguagem, estruturas de dados, controle de fluxo, funções, módulos e outras funcionalidades.

O tutorial oficial é uma introdução aos conceitos e recursos da linguagem e recomenda experiência básica de programação para seu público-alvo.

2.1 Conceitos fundamentais

Entre os conceitos fundamentais para iniciar em Python estão:

- variáveis e tipos de dados;
- operadores;
- estruturas condicionais, como if;
- estruturas de repetição, como for e while;
- funções;
- listas, tuplas e dicionários;
- módulos e pacotes;
- tratamento de exceções.

2.2 Exemplo de função

Uma função permite organizar uma tarefa em um bloco reutilizável de código.

```
def calcular_total(valores):  
    return sum(valores)  
  
valores = [100, 250, 80]  
total = calcular_total(valores)  
print(total)
```

3. Python aplicado a dados

Python pode ser utilizado para carregar, transformar, analisar e exportar dados.

Uma das bibliotecas mais utilizadas nesse contexto é o pandas, que fornece estruturas de dados e ferramentas para análise de dados.

3.1 Pandas

A documentação oficial do pandas descreve a biblioteca como uma ferramenta de estruturas de dados e análise de dados para Python.

Entre suas estruturas principais estão Series, de uma dimensão, e DataFrame, de duas dimensões.

O DataFrame é especialmente adequado para dados tabulares, como informações armazenadas em planilhas ou tabelas.

3.2 Operações comuns com dados

Em uma rotina de análise, algumas operações frequentes são:

- importar dados;
- visualizar registros;
- selecionar colunas;
- filtrar linhas;
- tratar dados ausentes;
- transformar valores;
- agrupar informações;
- realizar cálculos e agregações;
- combinar tabelas;
- exportar resultados.

3.3 Exemplo com pandas

O exemplo abaixo representa uma operação simples de leitura e análise de dados tabulares.

```
import pandas as pd

dados = pd.read_excel("pagamentos.xlsx")

pendentes = dados[dados["Status"] == "Pendente"]

total = pendentes["Valor"].sum()

print(pendentes)
print("Total pendente:", total)
```

4. Arquivos e Excel

O pandas oferece funções para integração com diferentes fontes de dados, incluindo CSV, Excel, SQL, JSON e Parquet.

As funções de leitura seguem, em muitos casos, o padrão `read_*`, como `read_csv()` e `read_excel()`.

4.1 OpenPyXL

OpenPyXL é uma biblioteca Python voltada à leitura e escrita de arquivos do Excel nos formatos baseados em Office Open XML, como XLSX e XLSM.

Ela permite criar e abrir workbooks, acessar planilhas e células, alterar valores e salvar arquivos.

4.2 Exemplo com OpenPyXL

```
from openpyxl import load_workbook

wb = load_workbook("relatorio.xlsx")
ws = wb.active

for row in ws.iter_rows(values_only=True):
    print(row)
```

5. Automação de dados

Automação consiste em utilizar código para executar tarefas de forma estruturada e repetível, reduzindo a necessidade de execução manual.

Em rotinas administrativas e de dados, exemplos podem incluir leitura de arquivos, validação de informações, consolidação de planilhas, cálculos, geração de relatórios e organização de arquivos.

5.1 Exemplo de fluxo automatizado

Um processo simples pode seguir estas etapas:

1. localizar o arquivo de entrada;
2. carregar os dados;
3. verificar a estrutura e a qualidade dos dados;
4. aplicar regras de transformação;
5. calcular indicadores;
6. gerar um arquivo de saída;
7. registrar ou comunicar o resultado.

6. Qualidade dos dados

Antes de produzir indicadores, é importante verificar problemas que podem afetar a análise, como valores ausentes, tipos de dados incorretos, duplicidades, inconsistências de preenchimento e informações fora do padrão esperado.

A limpeza e transformação dos dados devem ser documentadas para que o processo possa ser compreendido e reproduzido.

7. Exemplo integrado

Considere uma planilha com as colunas Fornecedor, Data, Valor, Status e Unidade.

Um processo Python poderia carregar essa planilha, verificar dados ausentes, filtrar registros pendentes, calcular o valor total por unidade e exportar um relatório para uma nova planilha.

```
import pandas as pd
```

```
dados = pd.read_excel("pagamentos.xlsx")

dados["Valor"] = pd.to_numeric(dados["Valor"], errors="coerce")

pendentes = dados[dados["Status"] == "Pendente"]

resumo = (
    pendentes
    .groupby("Unidade", as_index=False)["Valor"]
    .sum()
    .rename(columns={"Valor": "Valor_Pendente"})
)

resumo.to_excel("relatorio_pendencias.xlsx", index=False)
```

8. Boas práticas

Ao desenvolver scripts para análise e automação, é recomendável:

- utilizar nomes de variáveis claros;
- dividir tarefas complexas em funções;
- validar entradas;
- tratar erros previsíveis;
- evitar informações confidenciais diretamente no código;
- manter arquivos de entrada e saída organizados;
- documentar regras importantes;
- testar o processo com dados de exemplo antes de utilizá-lo em uma rotina real.

9. Fontes oficiais utilizadas

Python — documentação oficial:

<https://docs.python.org/pt-br/3/>

Python Tutorial:

<https://docs.python.org/pt-br/3/tutorial/>

Pandas — documentação oficial:

<https://pandas.pydata.org/docs/>

Pandas — Getting Started:

https://pandas.pydata.org/docs/getting_started/index.html

OpenPyXL — documentação oficial:

<https://openpyxl.readthedocs.io/en/stable/>

Observação: este material é uma síntese didática baseada nessas documentações. As páginas oficiais devem ser consultadas para detalhes, limitações, versões e referências completas.